



Poupa+

Aplicativo de gestão de finanças pessoais

Documento de Concepção, Requisitos e Prototipação

Requisitos

Escolha de plataforma

Wireframes & Mockups

Flutter e Android

Aluno Vinícios Andrei Mensen

Curso Análise e Desenvolvimento de Sistemas

Professor Alysson Borges

Turma 28743 - DESENVOLVIMENTO MOBILE - EAD54-12

Trabalho referente aos conteúdos da Semana 1

1 Visão Geral do Projeto

1.1 Contexto e problema identificado

O endividamento das famílias brasileiras é um problema antigo e que não dá sinais de melhora. Boa parte das pessoas não consegue dizer com precisão quanto gasta por mês, nem em quais categorias esse dinheiro é consumido. O motivo raramente é falta de vontade. O que atrapalha de verdade é a **fricção do registro**: planilhas exigem muita disciplina e nunca estão à mão na hora da compra, os aplicativos de banco mostram só o extrato daquela instituição e as ferramentas mais completas do mercado costumam ser complexas demais para quem está começando agora.

O resultado costuma ser sempre o mesmo: a pessoa começa a controlar os gastos, larga o registro em duas ou três semanas e volta a andar no escuro até o próximo aperto financeiro.

1.2 Solução proposta

O **Poupa+** é um aplicativo mobile de finanças pessoais construído em cima de uma ideia simples: **registrar um gasto precisa levar menos de dez segundos**. A partir desse registro rápido, o app organiza os lançamentos por categoria, compara o que foi gasto com o que estava planejado para o mês e traduz os números em uma resposta visual imediata para a pergunta que a pessoa de fato faz, que é se dá ou não para gastar naquele momento.

PROPOSTA DE VALOR

Simplicidade

Fluxo de lançamento em 3 toques, sem cadastros obrigatórios de conta bancária.

DIFERENCIAL

Orçamento vivo

Limite por categoria com alerta em 80% e bloqueio visual ao estourar.

RETENÇÃO

Metas

Objetivos de poupança com barra de progresso e projeção de data de conclusão.

1.3 Público-alvo e persona

Adultos entre 20 e 40 anos, com renda mensal de 1 a 5 salários mínimos, alta familiaridade com smartphones e baixa familiaridade com instrumentos formais de controle financeiro.

Atributo	Persona primária
Nome	Marina, 27 anos, analista administrativa
Rotina	Trabalha em regime híbrido, usa o celular como dispositivo principal e raramente abre um computador fora do expediente
Dor	"Meu salário some antes do fim do mês e eu não sei explicar em quê"
Objetivo	Enxergar para onde vai o dinheiro e conseguir guardar uma reserva de emergência
Barreira	Já tentou planilha e dois aplicativos; desistiu por acharem trabalhoso demais lançar cada compra
Dispositivo	Android intermediário (4 GB de RAM), plano de dados limitado, uso frequente sem conexão

1.4 Objetivos do produto

Objetivo geral: entregar uma ferramenta de controle financeiro pessoal que a pessoa consiga manter em uso por mais de 30 dias seguidos, sem largar no meio do caminho.

Objetivos específicos: reduzir o tempo de registro de uma despesa para menos de 10 segundos, deixar o saldo disponível visível logo na tela inicial, avisar antes do orçamento estourar e funcionar por completo mesmo sem conexão com a internet.

1.5 Método de levantamento de requisitos

Os requisitos foram elicitados a partir da combinação de três técnicas, conforme o conteúdo estudado na Semana 1:

Técnica	Aplicação no projeto	Resultado
Entrevista	Conversas semiestruturadas com 5 potenciais usuários do perfil da persona	Identificação da fricção de registro como causa principal do abandono
Benchmarking	Análise comparativa de 3 aplicativos concorrentes do segmento	Definição do escopo mínimo esperado e das oportunidades de simplificação
Prototipação	Construção de wireframes navegáveis validados com os entrevistados	Ajuste do fluxo de lançamento de 5 para 3 toques

2 Requisitos do Aplicativo

Os requisitos estão organizados em **funcionais** (o que o sistema faz), **não funcionais** (com que qualidade o sistema faz) e **regras de negócio** (restrições do domínio). A priorização segue o método MoSCoW, sinalizado na última coluna: **Essencial** deve estar na versão 1; **Importante** agrega valor relevante mas pode ser postergado; **Desejável** fica para versões futuras.

2.1 Requisitos funcionais

Código	Requisito	Descrição detalhada	Prior.
RF01	Cadastro e autenticação	O sistema deve permitir que o usuário crie uma conta com nome, e-mail e senha, e faça login posteriormente. Deve oferecer também recuperação de senha por e-mail e a opção de usar o app em modo local, sem conta.	Essencial
RF02	Configuração inicial	No primeiro acesso, o sistema deve conduzir um assistente que capta a renda mensal aproximada e as categorias de gasto que o usuário deseja acompanhar.	Essencial
RF03	Registro de despesa	O sistema deve permitir lançar uma despesa informando valor, categoria, data e uma descrição opcional. A data deve vir preenchida com o dia corrente e a categoria deve sugerir a última utilizada.	Essencial
RF04	Registro de receita	O sistema deve permitir lançar receitas (salário, freelance, outros) com valor, origem e data, distinguindo-as visualmente das despesas.	Essencial
RF05	Edição e exclusão	O sistema deve permitir editar ou excluir qualquer lançamento já registrado, solicitando confirmação antes da exclusão definitiva.	Essencial
RF06	Gestão de categorias	O sistema deve disponibilizar categorias padrão (alimentação, transporte, moradia, lazer, saúde, educação) e permitir que o usuário crie, renomeie, escolha ícone e cor, e desative categorias próprias.	Essencial
RF07	Painel de resumo	A tela inicial deve exibir, referente ao mês corrente, o saldo disponível, o total de receitas, o total de despesas e o percentual do orçamento já consumido.	Essencial
RF08	Extrato de lançamentos	O sistema deve listar todos os lançamentos agrupados por data, com filtros por período, categoria e tipo, além de busca textual pela descrição.	Essencial
RF09	Orçamento por categoria	O sistema deve permitir definir um teto de gasto mensal por categoria e exibir o consumo em barra de progresso comparando planejado e realizado.	Essencial
RF10	Alertas de orçamento	O sistema deve emitir notificação local quando o gasto de uma categoria atingir 80% do teto definido e novamente quando o ultrapassar.	Importante
RF11	Metas de economia	O sistema deve permitir criar metas com nome, valor-alvo e prazo, registrar aportes e exibir o progresso percentual e o valor mensal necessário para cumprir o prazo.	Importante

RF12	Relatórios visuais	O sistema deve gerar gráfico de rosca com a distribuição de despesas por categoria e gráfico de barras com a evolução mensal dos últimos seis meses.	Importante
RF13	Despesas recorrentes	O sistema deve permitir marcar um lançamento como recorrente (mensal ou semanal) e gerá-lo automaticamente na data prevista.	Importante
RF14	Exportação de dados	O sistema deve permitir exportar os lançamentos de um período em formato CSV e compartilhá-lo pelos aplicativos instalados no dispositivo.	Desejável
RF15	Sincronização em nuvem	O sistema deve sincronizar os dados entre dispositivos do mesmo usuário quando houver conexão, resolvendo conflitos pelo registro mais recente.	Desejável
RF16	Bloqueio biométrico	O sistema deve oferecer proteção da abertura do app por biometria ou PIN, utilizando o recurso nativo do aparelho.	Desejável

2.2 Requisitos não funcionais

Código	Categoria	Descrição e critério mensurável
RNF01	Desempenho	A tela inicial deve ser exibida com os dados carregados em no máximo 2 segundos em um aparelho Android intermediário com 4 GB de RAM.
RNF02	Usabilidade	O registro de uma despesa deve ser concluído em no máximo 3 toques a partir da tela inicial e em menos de 10 segundos.
RNF03	Disponibilidade offline	Todas as funcionalidades essenciais (RF03 a RF09) devem operar integralmente sem conexão com a internet, usando banco de dados local.
RNF04	Segurança	As senhas devem ser armazenadas com hash e sal; o tráfego com o servidor deve usar HTTPS/TLS 1.2 ou superior; dados sensíveis locais devem ficar em armazenamento criptografado.
RNF05	Privacidade	O tratamento de dados pessoais deve estar em conformidade com a LGPD, com consentimento explícito e opção de exclusão total da conta e dos dados.
RNF06	Compatibilidade	O app deve funcionar em Android 8.0 (API 26) ou superior e em iOS 13 ou superior, cobrindo mais de 90% dos aparelhos em uso.
RNF07	Responsividade	A interface deve se adaptar a telas de 4,7" a 6,9" sem truncamento de conteúdo nem rolagem horizontal.
RNF08	Acessibilidade	Contraste mínimo de 4,5:1 entre texto e fundo, rótulos compatíveis com leitores de tela e áreas de toque de no mínimo 48 por 48 dp.
RNF09	Confiabilidade	Nenhum lançamento pode ser perdido em caso de encerramento abrupto do app; a gravação local deve ser transacional.
RNF10	Manutenibilidade	O código deve seguir arquitetura em camadas com separação entre interface, regras de negócio e acesso a dados, com cobertura de testes automatizados de no mínimo 60% na camada de negócio.
RNF11	Consumo de recursos	O pacote de instalação deve ficar abaixo de 40 MB e o app não deve consumir mais de 150 MB de memória em uso normal.

RNF12	Localização	A interface deve estar em português do Brasil, com valores em real (R\$), separador decimal por vírgula e datas no formato dd/mm/aaaa.
--------------	-------------	--

2.3 Regras de negócio

Código	Regra
RN01	O valor de um lançamento deve ser sempre maior que zero; valores negativos são rejeitados na validação do formulário.
RN02	O saldo disponível do mês é calculado como: total de receitas do mês menos total de despesas do mês.
RN03	Um lançamento não pode ter data futura superior a 12 meses nem anterior à data de criação da conta.
RN04	Uma categoria com lançamentos vinculados não pode ser excluída, apenas desativada, preservando o histórico.
RN05	O orçamento é sempre mensal e reinicia automaticamente no primeiro dia de cada mês.
RN06	Uma meta é considerada concluída quando a soma dos aportes atinge ou supera o valor-alvo, e passa a ficar somente leitura.
RN07	Aportes em metas são contabilizados como despesa da categoria "Poupança" para não inflar artificialmente o saldo disponível.

2.4 Histórias de usuário

ID	História	Critérios de aceite
US01	Como usuária, quero registrar um gasto em poucos toques para não abandonar o controle no dia a dia.	Botão de ação visível na tela inicial; data e categoria pré-preenchidas; salvamento com confirmação visual em até 1 segundo.
US02	Como usuária, quero ver quanto ainda posso gastar no mês para decidir uma compra na hora.	Saldo disponível em destaque na primeira dobra da tela inicial, sem necessidade de rolagem.
US03	Como usuária, quero ser avisada antes de estourar o orçamento de uma categoria.	Notificação local disparada ao atingir 80% do teto; barra da categoria muda de cor no painel.
US04	Como usuária, quero visualizar em que categorias gastei mais no mês para saber onde cortar.	Gráfico de rosca com percentuais e lista ordenada por valor decrescente.
US05	Como usuária, quero acompanhar o progresso da minha reserva de emergência para me manter motivada.	Barra de progresso, percentual concluído e valor mensal necessário para cumprir o prazo.
US06	Como usuária, quero usar o app mesmo sem internet, porque nem sempre tenho dados disponíveis.	Lançamento, consulta e edição funcionam em modo avião; sincronização ocorre quando a conexão retorna.

2.5 Escopo negativo (fora da versão 1)

Delimitar o que **não** será feito é parte do requisito. Ficam explicitamente fora do escopo desta versão: integração automática com bancos via Open Finance; controle de investimentos e renda variável; gestão financeira compartilhada entre múltiplos usuários; emissão de nota fiscal ou funcionalidades de pessoa jurídica; e leitura automática de cupons fiscais por câmera. Essas

funcionalidades foram mapeadas como evolução futura por exigirem homologação regulatória, parcerias ou custo de infraestrutura incompatíveis com um MVP acadêmico.

3 Escolha da Plataforma de Desenvolvimento

3.1 Alternativas avaliadas

Foram consideradas quatro rotas tecnicamente viáveis para o projeto, cobrindo as três abordagens apresentadas na Semana 1, que são o desenvolvimento nativo, o desenvolvimento híbrido e a escolha por uma plataforma única:

Alternativa	Tecnologia	Característica central
A. Nativo Android	Kotlin + Jetpack Compose	Código exclusivo para Android, com acesso direto e imediato a toda a API do sistema
B. Nativo iOS	Swift + SwiftUI	Código exclusivo para iOS, exige macOS e conta de desenvolvedor Apple
C. Híbrido (Flutter)	Dart + Flutter	Base única de código compilada para binário nativo, com motor gráfico próprio
D. Híbrido (React Native)	JavaScript/TypeScript	Base única de código que renderiza componentes nativos por meio de uma ponte

3.2 Comparativo qualitativo

Critério	Nativo Android	Nativo iOS	Flutter	React Native
Bases de código	1 por plataforma	1 por plataforma	Única	Única
Sistema exigido para compilar	Windows, Linux ou macOS	Somente macOS	Windows, Linux ou macOS (Android); macOS para iOS	Igual ao Flutter
Emulador/simulador	Android Emulator (AVD), multiplataforma	iOS Simulator, exclusivo do Xcode	Android Emulator + hot reload	Android Emulator + fast refresh
Desempenho de UI	Máximo	Máximo	Alto (compila para código nativo)	Bom (custo da ponte JS)
Consistência visual entre SOs	N/A	N/A	Alta (renderização própria)	Média (componentes nativos divergem)
Recursos nativos (biometria, notificações)	Direto	Direto	Via plugins maduros	Via bibliotecas de terceiros
Esforço estimado do MVP	Alto (dobra se houver iOS)	Alto	Reduzido	Reduzido
Curva de aprendizado	Média	Média/alta	Média (Dart é nova, mas regular)	Baixa para quem já usa JS

3.3 Matriz de decisão ponderada

Para evitar uma escolha baseada em preferência pessoal, os critérios receberam pesos de 1 a 5 conforme sua relevância para **este** projeto, e cada alternativa foi pontuada de 1 a 5. A nota final é a soma dos produtos peso × pontuação.

Critério de decisão	Peso	Nativo Android	Nativo iOS	Flutter	React Native
Alcance do público-alvo no Brasil	5	5	2	5	5
Custo e prazo de desenvolvimento	5	2	2	5	4
Ambiente de testes disponível (emulador)	4	5	1	5	5
Reaproveitamento de código entre plataformas	4	2	2	5	4
Desempenho e fluidez da interface	3	5	5	4	3
Acesso a recursos nativos do aparelho	3	5	5	4	4
Maturidade do ecossistema e documentação	3	5	5	4	5
PONTUAÇÃO FINAL (máx. 135)		108	77	126	117

Decisão: desenvolvimento **híbrido com Flutter**, tendo o **Android como plataforma-alvo primária** para validação e testes no emulador, e o iOS como destino secundário a partir da mesma base de código.

3.4 Fundamentação da escolha

a) Alcance sobre o público-alvo

A persona definida usa um aparelho Android intermediário, e esse perfil é o que domina o mercado brasileiro. Levantamentos recentes de participação de sistemas operacionais móveis colocam o Android acima de 85% no Brasil, contra algo próximo de 70% na média mundial. Para um aplicativo voltado justamente à faixa de renda mais sensível a preço, priorizar o Android é uma decisão de produto antes de ser uma decisão técnica. Por outro lado, abandonar o iOS de vez seria arriscado, porque os usuários de iPhone respondem por boa parte do gasto mundial com aplicativos, e isso passa a pesar caso o produto vire um modelo pago. A abordagem híbrida resolve os dois lados de uma vez: entrega primeiro onde o público está e mantém o caminho aberto para o iOS sem reescrever o aplicativo do zero.

b) Viabilidade do ambiente de desenvolvimento e testes

Este é o critério decisivo do contexto acadêmico. O desenvolvimento nativo para iOS exige, obrigatoriamente, um computador com macOS para executar o Xcode e o iOS Simulator, restrição que na prática elimina a alternativa B para a maior parte dos estudantes. O **Android Emulator**, ao contrário, roda em Windows, Linux e macOS, é gratuito e permite configurar diferentes tamanhos de tela, versões de API e condições de rede, atendendo diretamente aos requisitos RNF06 e RNF07. Como o Flutter compila para Android usando exatamente esse mesmo emulador, o projeto pode ser integralmente desenvolvido, testado e demonstrado sem nenhum investimento em hardware adicional.

c) Custo, prazo e produtividade

Manter duas bases nativas significa, na prática, escrever e testar o mesmo aplicativo duas vezes, com duas linguagens e dois conjuntos de bibliotecas. Com o Flutter, uma única base de código em Dart gera os binários para ambas as plataformas, e o recurso de *hot reload* aplica alterações de interface em segundos, sem recompilação completa. Em um projeto com prazo curto e equipe reduzida, esse ganho de produtividade é o que torna o escopo dos 16 requisitos funcionais realista.

d) Adequação técnica ao produto

O Poupa+ é um aplicativo centrado em formulários, listas, gráficos e banco de dados local. Não é um jogo 3D, não é um editor de vídeo e não depende de processamento pesado de sensores. Nesse tipo de aplicação, a diferença de desempenho entre Flutter e nativo passa despercebida pelo usuário. Além disso, o Flutter compila para código nativo ARM e desenha a interface com motor gráfico próprio, o que garante que o layout dos wireframes da seção 4 apareça **igual** em qualquer aparelho. Essa é uma vantagem direta sobre o React Native, que delega a renderização aos componentes nativos de cada sistema e acaba gerando variações visuais indesejadas em um app cuja identidade depende de gráficos e cores.

e) Cobertura dos requisitos não funcionais

Requisito	Exigência	Como o Flutter atende
RNF01	Carregamento em até 2 s	Compilação AOT para código nativo ARM, sem interpretador em tempo de execução
RNF03	Operação offline	Persistência local com SQLite via pacotes <i>sqflite</i> ou <i>drift</i>
RNF04	Armazenamento seguro	Pacote <i>flutter_secure_storage</i> , que usa Keystore no Android e Keychain no iOS
RNF07	Adaptação a telas de 4,7" a 6,9"	Widgets responsivos (<i>LayoutBuilder</i> , <i>MediaQuery</i> , <i>Flex</i>) testáveis em múltiplos AVDs
RNF08	Acessibilidade WCAG AA	<i>Semantics</i> nativo do framework, integrado a TalkBack e VoiceOver
RNF11	APK abaixo de 40 MB	Geração de <i>app bundle</i> com divisão por arquitetura e remoção de código não utilizado

3.5 Stack técnica definida

Camada	Tecnologia adotada
Linguagem e framework	Dart 3 + Flutter 3 (canal estável)
Arquitetura	Camadas com padrão MVVM e injeção de dependências
Gerência de estado	Provider ou Riverpod
Persistência local	SQLite (<i>sqflite</i>) para lançamentos; <i>shared_preferences</i> para preferências
Gráficos	<i>fl_chart</i> (rosca e barras)
Notificações	<i>flutter_local_notifications</i> (alertas de orçamento, RF10)
Backend opcional (RF15)	Firebase Authentication + Cloud Firestore
IDE e testes	Android Studio + Android Emulator (AVD Pixel 6, API 34)
Prototipação	Figma para wireframes e mockups de média fidelidade
Versionamento	Git + GitHub

3.6 Riscos e mitigações

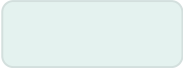
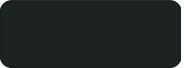
Risco	Impacto	Mitigação
-------	---------	-----------

Curva de aprendizado do Dart para a equipe	Médio	Reservar a primeira semana para estudo dirigido; usar a documentação oficial e o catálogo de widgets
Dependência de pacotes de terceiros sem manutenção	Médio	Adotar apenas pacotes com selo <i>Flutter Favorite</i> ou atualização nos últimos 6 meses
Emulador lento em máquinas sem virtualização	Alto	Habilitar aceleração por hardware; alternativamente, testar em aparelho físico via depuração USB
Impossibilidade de gerar o build iOS sem macOS	Baixo	Escopo da entrega limitado ao Android; build iOS posterior via serviço de integração contínua em nuvem
Escopo maior que o prazo disponível	Alto	Priorização MoSCoW já aplicada: entregar apenas os requisitos essenciais na versão 1

4 Protótipo: Wireframes e Mockups

O protótipo foi construído em **média fidelidade**: a estrutura, a hierarquia da informação e o fluxo de navegação estão definidos com precisão, e a identidade visual (paleta, tipografia e componentes) já está aplicada, mas sem o refinamento gráfico final. Esse nível é o mais adequado à etapa de concepção, pois permite validar o fluxo com usuários antes de investir em arte final.

4.1 Sistema de design

						
#0D6E62 Primária	#08483F Primária escura	#E4F2EF Superfície	#1F7A3F Receita	#B3261E Despesa / estouro	#E08B00 Alerta 80%	#1C2321 Texto

Racional cromático: o verde-petróleo passa estabilidade e é culturalmente associado a dinheiro, sem o tom agressivo do vermelho bancário. As cores semânticas aparecem só para indicar estado (receita, despesa, alerta) e nunca como decoração, de modo que a cor sempre carrega informação. **Tipografia:** família Inter, com escala de 12/14/16/22 sp e peso 700 reservado a valores monetários. **Espaçamento:** grade de 8 dp. **Toque:** alvos de no mínimo 48×48 dp, atendendo ao RNF08.

4.2 Fluxo de navegação



A arquitetura adota **navegação por abas inferiores** com profundidade máxima de dois níveis: nenhuma funcionalidade essencial exige mais de dois toques a partir do painel. O botão de ação flutuante permanece visível em todas as abas, garantindo o cumprimento do RNF02.

4.3 Telas do protótipo



Tela 1. Login

Atende ao RF01. O modo local sem cadastro tira a barreira de entrada. Campos com rótulo persistente acima, não apenas *placeholder*, por acessibilidade. Ação primária e secundária visualmente hierarquizadas.



Tela 2. Configuração inicial

Atende ao RF02 e ao RF06. O indicador de progresso reduz o abandono no *onboarding*. Seleção por *chips* em vez de formulário longo. Opção de pular preserva a autonomia de quem quer usar o app imediatamente.



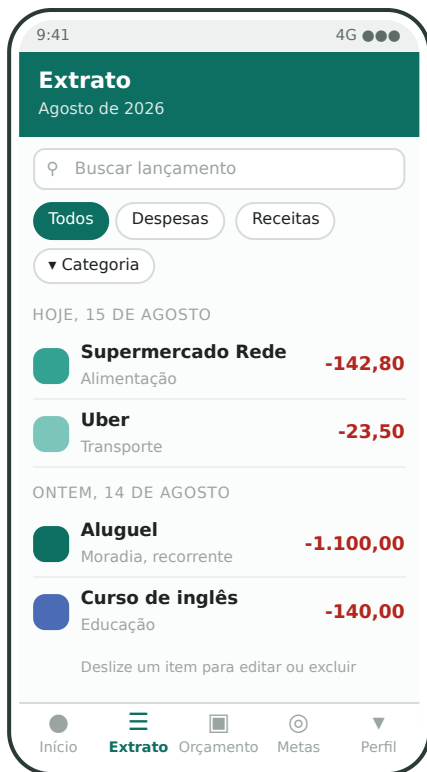
Tela 3. Painel (Home)

Atende ao RF07 e às histórias US02 e US03. O saldo disponível ocupa a primeira dobra, sem rolagem (RNF02). Barra de orçamento agregada responde à pergunta "posso gastar hoje?". Botão flutuante de lançamento sempre acessível.



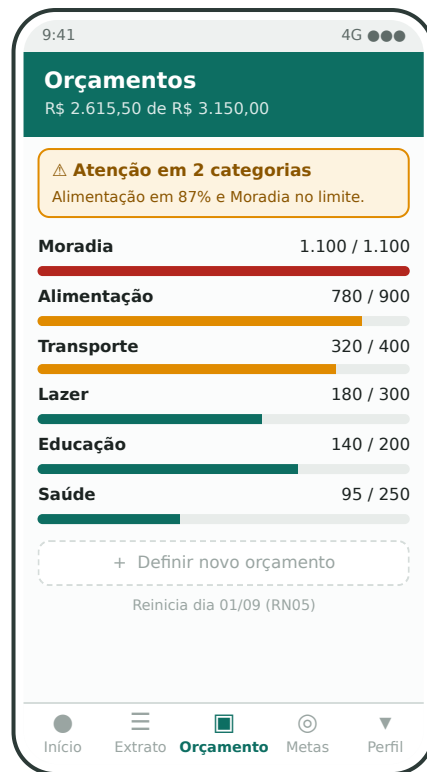
Tela 4. Nova transação

Atende ao RF03, RF04 e à história US01. O teclado numérico é próprio, já que o valor é o campo mais importante e recebe foco automático. Data preenchida com o dia corrente e categoria sugerida pelo último uso (RN01, RN03). Fluxo de 3 toques: valor → categoria → salvar.



Tela 5. Extrato

Atende ao RF05 e RF08. O agrupamento por dia com cabeçalhos fixos facilita a leitura cronológica. Filtros por tipo e categoria em *chips*, sem sair da tela. Edição e exclusão por gesto de deslizar, com confirmação.



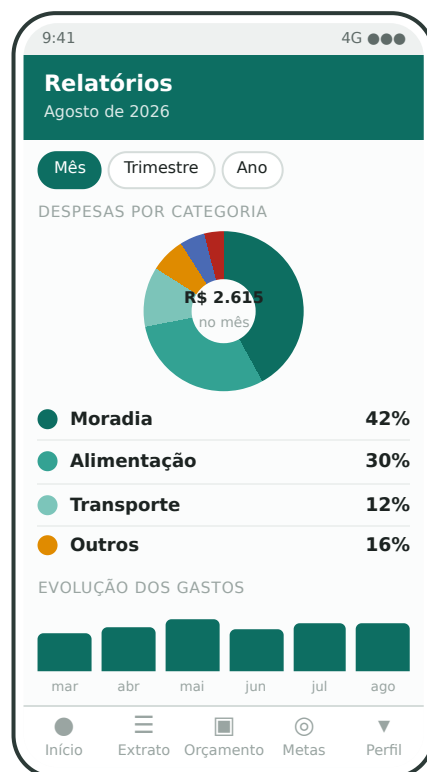
Tela 6. Orçamentos

Atende ao RF09, RF10 e à história US03. Semáforo de cores: verde até 79%, âmbar de 80% a 99%, vermelho ao atingir o teto. O aviso no topo aparece antes do estouro acontecer, e não depois. Valores absolutos e barra aparecem juntos, então a informação não depende só da cor.



Tela 7. Metas

Atende ao RF11, RN06, RN07 e à história US05. O valor que precisa ser guardado por mês já vem calculado, o que ajuda a manter o ritmo. Metas concluídas continuam visíveis, em modo somente leitura, como reforço positivo.



Tela 8. Relatórios

Atende ao RF12 e à história US04. O gráfico de rosca traz o valor total no centro, sem obrigar o usuário a estimar. Legenda ordenada por valor decrescente indica onde cortar primeiro. A série de seis meses mostra a tendência dos gastos ao longo do tempo.



Tela 9. Perfil e configurações

Atende ao RF14, RF15, RF16, RNF05 e RNF12. A exclusão total da conta fica na própria tela, cumprindo o direito de eliminação previsto na LGPD. Ação destrutiva sinalizada em vermelho e isolada do restante da lista.

4.4 Rastreabilidade requisito → tela

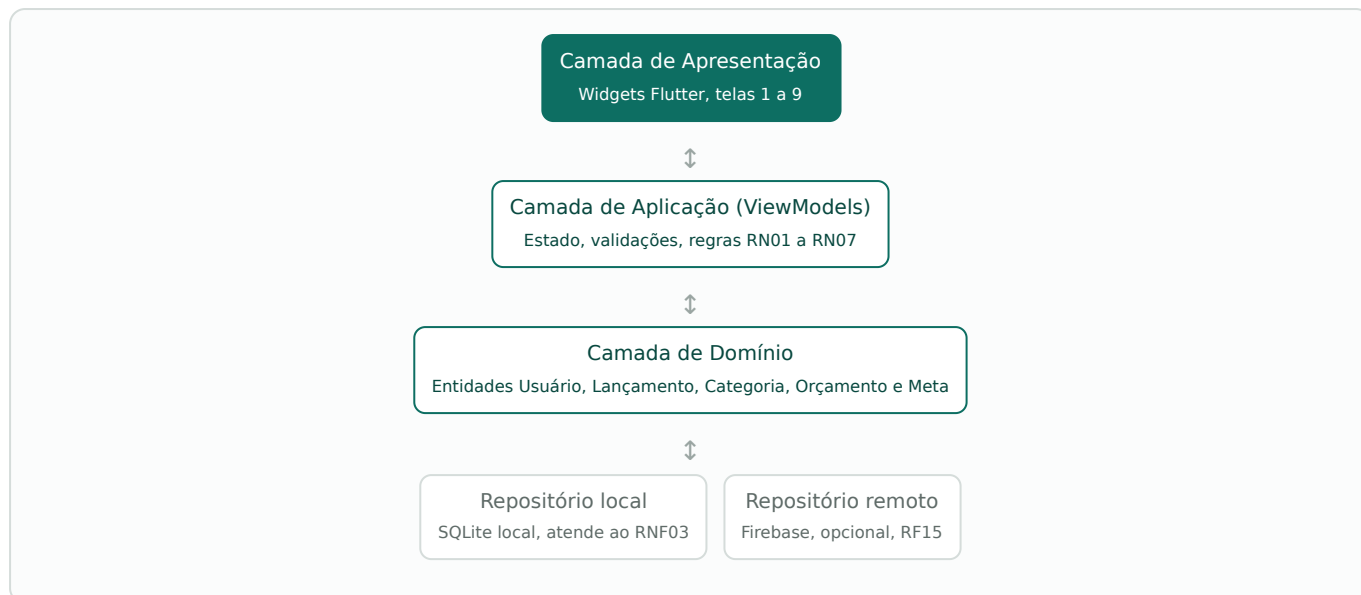
Tela	Requisitos cobertos
1. Login	RF01
2. Onboarding	RF02, RF06
3. Painel	RF07, RNF01, RNF02
4. Nova transação	RF03, RF04, RF13, RN01, RN03
5. Extrato	RF05, RF08
6. Orçamentos	RF09, RF10, RN05
7. Metas	RF11, RN06, RN07
8. Relatórios	RF12
9. Perfil	RF14, RF15, RF16, RNF05

A matriz acima mostra que **todos os 16 requisitos funcionais** têm representação no protótipo e que nenhuma tela foi desenhada sem requisito correspondente, o que dá um critério objetivo para avaliar se a prototipação está completa.

Próximo passo: transformar estes wireframes em protótipo navegável no Figma e submetê-lo a um teste de usabilidade com 5 usuários, medindo o tempo real de conclusão da tarefa "registrar uma despesa" contra a meta de 10 segundos do RNF02.

5 Viabilidade Técnica do Projeto

5.1 Arquitetura da solução



A separação em camadas atende diretamente ao RNF10 e permite trocar o mecanismo de persistência, saindo do SQLite local para a nuvem, sem mexer na interface, já que as duas implementações respeitam o mesmo contrato de repositório.

5.2 Modelo de dados simplificado

Entidade	Atributos principais	Relacionamentos
usuario	id, nome, email, senha_hash, renda_mensal, criado_em	1:N com lançamento, meta e orçamento
categoria	id, nome, icone, cor, tipo (despesa/receita), ativa	1:N com lançamento e orçamento
lançamento	id, usuario_id, categoria_id, valor, tipo, data, descricao, recorrente, sincronizado	N:1 com usuário e categoria
orcamento	id, usuario_id, categoria_id, valor_teto, mes_referencia	N:1 com usuário e categoria
meta	id, usuario_id, nome, valor_alvo, valor_atual, prazo, concluida	1:N com aporte
aporte	id, meta_id, valor, data	N:1 com meta

5.3 Cronograma estimado do MVP

Etapas	Duração	Entregas
1. Concepção	1 semana	Levantamento de requisitos, definição de personas, escolha da plataforma (este documento)
2. Prototipação	1 semana	Wireframes, sistema de design, protótipo navegável e teste de usabilidade
3. Setup técnico	3 dias	Projeto Flutter, configuração do emulador (AVD), estrutura de pastas, repositório Git

4. Núcleo	2 semanas	RF01 a RF08: autenticação, banco local, CRUD de lançamentos, painel e extrato
5. Orçamento e metas	1,5 semana	RF09 a RF12: tetos por categoria, notificações, metas e gráficos
6. Testes e ajustes	1 semana	Testes unitários e de widget, validação em múltiplos AVDs, correções de acessibilidade
7. Empacotamento	3 dias	Geração do <i>app bundle</i> , ícone, tela de abertura e material de publicação
Total estimado		Aproximadamente 8 semanas para a versão 1 (requisitos essenciais e importantes)

5.4 Indicadores de sucesso

Indicador	Meta	Como medir
Retenção em 30 dias	$\geq 35\%$	Percentual de usuários que registram ao menos um lançamento após 30 dias
Tempo de registro de despesa	$\leq 10\text{ s}$	Cronometragem em teste de usabilidade moderado com 5 participantes
Lançamentos por usuário/semana	≥ 7	Evento de analytics disparado a cada gravação bem-sucedida
Taxa de falhas	$< 1\%$	Sessões sem <i>crash</i> monitoradas por ferramenta de relatório de erros

5.5 Considerações finais

O projeto do **Poupa+** passou por todas as etapas iniciais propostas na atividade. Partiu de um problema real e bem delimitado, transformou esse problema em requisitos funcionais e não funcionais mensuráveis, escolheu a plataforma com base em uma matriz de decisão montada sobre critérios objetivos e chegou a nove telas de protótipo, cada uma ligada aos requisitos que a originaram.

A escolha pelo desenvolvimento híbrido com Flutter, tendo o Android como alvo principal, é a que melhor concilia as três restrições do contexto. O público-alvo brasileiro está majoritariamente no Android, o ambiente de desenvolvimento e testes por emulador está disponível sem custo e sem exigência de hardware específico, e a base única de código deixa a porta aberta para o iOS sem jogar fora o trabalho já feito. É, ao mesmo tempo, uma decisão tecnicamente sólida e viável dentro do escopo proposto.